

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Компьютерные науки и анализ данных»

Отчет о программном проекте на тему:

PlagLens: платформа проверки программных задач

Выполнил студент:

группы БКНАД241, 2 курс
Смирнов Максим Сергеевич

Принял руководитель проекта:

Горшков Сергей Сергеевич
Приглашенный преподаватель
Факультет компьютерных наук НИУ ВШЭ

Содержание

Аннотация	4
Ключевые слова	4
1 Введение	5
2 Обзор литературы и аналогов	6
2.1 Классы подходов к детектированию заимствований в исходном коде	6
2.2 Системы-аналоги в образовательной практике	6
2.3 Подходы к обнаружению и интерпретации использования LLM	7
3 Постановка задачи и требования	7
3.1 Роли пользователей и основные сценарии	7
3.2 Функциональные требования	7
3.3 Нефункциональные требования	8
4 Архитектура и проектирование системы	8
4.1 Общая архитектура	8
4.2 Компоненты и границы ответственности	9
4.3 Синхронные и асинхронные взаимодействия	10
4.4 Основные потоки данных	10
5 Реализация ключевых модулей и интеграций	11
5.1 Технологический стек и организация кода	11
5.2 Модель данных (проектное предложение)	12
5.3 Примерная спецификация API (проектное предложение)	12
5.4 Модуль проверки на заимствования	13
5.5 Модуль LLM-анализа кода	14
5.6 Интеграции со сторонними сервисами	14
6 Безопасность и управление секретами	15
6.1 Аутентификация и авторизация	15
6.2 Хранение секретов	15
6.3 Конфиденциальность и защита данных	15
7 Наблюдаемость, мониторинг и уведомления	15
7.1 Метрики и мониторинг	16
7.2 Логи и трассировка	16
8 Тестирование	16
8.1 Уровни тестирования	16
8.2 Критические сценарии	16

8.3 Ограничения результатов	17
9 План выполнения работ и связь с роадмапом	17
10 Заключение	18
Список литературы	19
А Список сокращений	20
Б Пример формата отчёта по заимствованиям (проектное допущение)	20

Аннотация

PlagLens — это платформа для автоматизации проверки программных заданий в учебных курсах. Она позволяет сократить время, которое преподаватель тратит на рутинные операции: сбор решений из внешних систем, поиск заимствований, выявление подозрительных фрагментов кода и формирование обратной связи студентам.

В основе проекта лежат два взаимодополняющих модуля: (1) модуль проверки на заимствования, реализованный как интеграция с внешним сервисом сравнения программ по API, и (2) модуль интеллектуального анализа, использующий внешнюю LLM по API для анализа кода, генерации пояснений и сигналов риска.

Система проектируется как набор слабо связанных сервисов вокруг доменной модели «курс—задание—посылка». В архитектуре явно выделены интеграционные адаптеры, сервисы анализа, подсистема уведомлений и подсистема экспорта результатов. Для хранения данных используется PostgreSQL, для выполнения фоновых задач — очередь (например, Celery с брокером Redis), а для мониторинга — Prometheus и Grafana.

Ключевые слова

Антиплагиат, проверка решений по программированию, winnowing, шинглы, нормализация кода, сравнение программных решений, подсветка совпадений (diff), интеграции Stepik и Яндекс.Контекст, LLM-анализ кода, FastAPI/PostgreSQL/Celery

1. Введение

Проверка программных заданий в массовых курсах — это трудоёмкий и чувствительный к ошибкам процесс. Преподавателю нужно собрать решения с разных платформ, сопоставить попытки студентов, найти заимствования и дать понятную обратную связь. С распространением генеративных моделей задача стала сложнее: в работах встречаются как взаимные копирования, так и фрагменты, сгенерированные LLM. В результате возникает потребность в инструменте, который автоматически собирает решения, анализирует код и наглядно показывает результаты проверки.

Объект исследования — процесс проверки программных заданий в образовательных курсах.

Предмет исследования — архитектурные и технические решения, обеспечивающие автоматизацию импорта решений, выявления сходства и формирование аналитической обратной связи.

Цель работы — разработать и описать программную платформу PlagLens для автоматизированной проверки решений на заимствования и признаки использования LLM.

Для достижения цели поставлены следующие задачи:

1. Провести обзор подходов к проверке сходства программ и практики применения таких систем в образовательной среде;
2. Определить пользовательские роли и сценарии, а также сформулировать функциональные и нефункциональные требования к платформе;
3. Спроектировать модульную архитектуру платформы и описать ключевые компоненты, включая хранение данных и мониторинг;
4. Описать API-интеграции с внешними источниками решений, сервисами проверки заимствований и LLM;
5. Предложить модель данных и примерные контракты API, достаточные для реализации базовых пользовательских сценариев;
6. Описать меры безопасности, принципы управления секретами, подход к наблюдаемости и методику тестирования.

Ожидаемые результаты включают архитектурную модель платформы, описание модулей и потоков данных, а также спецификации интерфейсов и форматов отчётов.

Отчёт построен следующим образом. В разделе 2 рассматриваются аналогичные решения и научные и практические подходы. В разделе 3 описаны требования и пользовательские сценарии. В разделе 4 описана архитектура PlagLens и взаимодействие компонентов. В разделе 5 описаны детали реализации ключевых модулей и интеграций (в части, подтверждаемой исходными материалами), а также предложения по контрактам API и форматам данных. В разделах 6 и 7 рассмотрены вопросы безопасности и наблюдаемости. В разделе 8 описана методика тестирования. В разделе 9 показана связь с планом работ. Заключение подводит итоги и отмечает ограничения и направления развития.

2. Обзор литературы и аналогов

2.1. Классы подходов к детектированию заимствований в исходном коде

Задача выявления заимствований в программном коде обычно решается методами, которые можно разделить на несколько классов.

Текстовые и токен-ориентированные методы приводят код к нормализованному виду (удаление комментариев, выравнивание пробелов, переименование идентификаторов), после чего вычисляют степень сходства строк, токенов или последовательностей. К этой группе также относится подход на основе отпечатков документа (fingerprinting) и шинглов, который позволяет эффективно сравнивать большое число документов «все со всеми» при разумной вычислительной сложности. Классическим примером является алгоритм winnowing [10].

Синтаксические методы основаны на абстрактных синтаксических деревьях (AST) и сравнивают деревья или их подструктуры. Такие методы более устойчивы к перестановкам операторов и локальным преобразованиям, но как правило, сложнее в реализации и требуют парсинга, зависящего от языка.

Семантические методы сравнивают программы по их поведению: по трассам исполнения, результатам тестов, графам потока управления и данным. Эти методы потенциально более устойчивы к переформулировкам, однако требуют развитой среды исполнения и набора тестов, а также связаны с рисками безопасности (выполнение недоверенного кода).

С практической стороны для образовательных курсов часто используются гибридные решения, сочетающие нормализацию, токенизацию и быстрые эвристики для отбора кандидатов.

2.2. Системы-аналоги в образовательной практике

В образовательной среде часто используются специализированные сервисы для проверки сходства программ, которые выдают результат в виде списка подозрительных пар и визуализированных совпадений. Классическим примером является **MOSS** [1], который обычно применяется как «чёрный ящик»: на вход подаётся набор решений, а на выходе формируется перечень пар с оценкой сходства и ссылками на совпадающие фрагменты.

Другим распространённым представителем этого класса является **JPlag** [9], поддерживающий несколько языков и различные стратегии нормализации. Помимо внешних сервисов, существуют локально разворачиваемые решения и инструменты, однако их использование требует поддержки вычислительной среды и настройки правил нормализации под конкретный курс, что усложняет сопровождение в учебном процессе.

Для PlagLens важно сочетание двух свойств: (I) возможность интеграции с внешними платформами, на которых ведётся обучение, и (II) прозрачность результатов, включая объяснимые сегменты совпадений, наглядную визуализацию и учёт контекста задания.

2.3. Подходы к обнаружению и интерпретации использования LLM

Надёжная автоматическая детекция использования LLM в программном коде остаётся открытой практической задачей. В отличие от плагиата между студентами, «источник» LLM отсутствует среди сдаваемых решений, а стилистические признаки сильно зависят от уровня подготовки студента и особенностей формулировки задания.

В прикладных системах вместо бинарной детекции целесообразно опираться на **сигналы риска** и их **интерпретацию**: поиск нетипичных конструкций, несоответствий принятому стилю курса и резких скачков сложности; сопоставление кода с «типовыми» решениями и выявление чрезмерной универсальности; генерация объяснений и вопросов, помогающих проверить понимание решения.

В этом контексте PlagLens рассматривает интеграцию с LLM как вспомогательный инструмент, дополняющий контур детектирования заимствований и повышающий качество обратной связи (например, за счёт автоматического резюме решения и подсветки потенциально проблемных мест). Такая интеграция может быть реализована через внешнее API.

3. Постановка задачи и требования

3.1. Роли пользователей и основные сценарии

Система PlagLens рассчитана на три базовые роли: **администратор**, **преподаватель** и **студент**. Администратор отвечает за настройки платформы, параметры интеграций, политики хранения данных, перечень курсов и права доступа. Преподаватель создаёт и ведёт курсы и задания, запускает импорт решений и проверки, анализирует отчёты, формирует оценки и предоставляет обратную связь. Студент просматривает статус проверок, результаты по своим решениям и полученную обратную связь.

К ключевым пользовательским сценариям относятся создание курса и задания, настройка источника данных (например, Stepik или Яндекс.Контест), импорт посылок по расписанию или по запросу, запуск проверки на заимствования и LLM-анализа для выбранного задания, просмотр преподавателем списка подозрительных пар или групп с детализацией совпадений, формирование итоговой оценки и комментариев с последующим экспортом результатов и отправкой уведомлений, а также просмотр студентом итогов проверки и соответствующих объяснений.

3.2. Функциональные требования

Функциональные требования задаются как набор возможностей, ожидаемых от платформы. Поскольку исходные материалы описывают целевую архитектуру и роадмап разработки, часть требований фиксируется как **планируемая функциональность**. Функциональность PlagLens включает управление пользователями и ролями, ведение курсов и заданий, привязку студентов и хранение посылок. Отдельный слой интеграций отвечает за получение данных из внешних образовательных платформ и обработку входящих событий.

Проверка на заимствования реализуется как запуск сравнения во внешнем сервисе по API с последующим сохранением и отображением отчётов. Контур LLM-анализа дополняет проверку сходства: для каждой посылки может формироваться структурированный отчёт (резюме, сигналы риска, вопросы для проверки понимания, рекомендации) через API сменяемого провайдера.

В качестве вспомогательной функциональности предусматриваются экспорт и отчётность (например, выгрузка оценок в таблицы), доставка уведомлений в интерфейс и внешние каналы, а также аудит действий пользователей и системных событий.

3.3. Нефункциональные требования

Нефункциональные требования описывают эксплуатационные свойства системы. Производительность обеспечивается асинхронной обработкой ресурсоёмких задач (импорт, проверки, анализ) при сохранении приемлемого времени ответа API. Масштабируемость достигается за счёт горизонтального расширения сервисов и воркеров фоновых задач.

Надёжность предполагает идемпотентность импорта и обработки вебхуков, а также устойчивость к временной недоступности внешних API. Безопасность включает разграничение доступа, защиту секретов интеграций и безопасную обработку кода как недоверенных данных.

Наблюдаемость обеспечивается сбором метрик, логов и алертов для ключевых потоков (импорт, проверки, экспорт) [2, 8]. Поддерживаемость достигается модульной структурой сервисов, понятными контрактами и документацией API.

4. Архитектура и проектирование системы

4.1. Общая архитектура

Архитектура PlagLens построена на разделении ответственности между уровнями. На уровне доступа располагаются Web Frontend и BFF/API Gateway, которые отвечают за аутентификацию и авторизацию, агрегацию запросов и формирование представлений, удобных для пользовательского интерфейса. Доменный уровень инкапсулирует бизнес-логику, связанную с курсами, заданиями и пользователями. Интеграционный уровень реализует адаптеры для внешних платформ и преобразование данных.

Ресурсоёмкие операции вынесены в отдельные вычислительные сервисы анализа, которые выполняются асинхронно и взаимодействуют с внешним сервисом проверки заимствований и с внешним LLM по API. Дополнительно выделены контуры отчётности, уведомлений и наблюдаемости, обеспечивающие необходимые эксплуатационные свойства системы.

На рисунке 4.1 показана сводная схема взаимодействия компонентов системы.

Processing Services отвечают за оркестрацию ресурсоёмких проверок и взаимодействие с внешними провайдерами.

Plagiarism Service инициирует проверки на заимствования во внешнем сервисе, подготавливает входные данные, принимает результаты (polling или webhook) и сохраняет отчёты.

AI Analysis Service выполняет запросы к внешнему LLM через сменяемого провайдера по API и сохраняет структурированный результат анализа.

Integration Services (адаптеры) инкапсулируют протоколы и особенности внешних образовательных платформ: Stepik API [11] и Яндекс.Контест (варианты интеграции) [13]. В качестве отдельных внешних каналов также выделяются Telegram Bot API [12] и Google Sheets API [6].

Notification Service агрегирует события (например, завершение импорта или публикацию оценок) и доставляет сообщения в интерфейс пользователя (SSE/WebSocket) либо во внешние каналы.

Reporting Services включают экспорт данных (таблицы и файлы), построение дашбордов и ведение журнала аудита.

Data Stores представлены PostgreSQL как основным хранилищем метаданных, результатов проверок и журнала событий [7].

Vault/Secrets (концептуально) используется для хранения секретов интеграций: ключей внешних API, токенов ботов и ключей сервисных аккаунтов.

Observability сбор метрик с помощью Prometheus и их визуализацию в Grafana [2, 8].

4.3. Синхронные и асинхронные взаимодействия

Синхронные взаимодействия реализуются по HTTP(S) и подходят для операций чтения, управления сущностями и запуска фоновых задач. Асинхронные взаимодействия используются для ресурсоёмких операций, таких как импорт посылок, сравнение большого числа решений и вызовы внешнего LLM API. Для асинхронной обработки целесообразно применять очередь задач, например Celery [3] с брокером Redis [5].

На диаграмме также выделена **шина событий** (Kafka). При необходимости она может использоваться для развязки контуров «доменные события — уведомления — отчётность». В минимальной конфигурации роль событийной коммуникации может выполнять очередь задач, а Kafka рассматривается как расширение для масштабирования и независимого потребления событий несколькими сервисами.

4.4. Основные потоки данных

Ниже приведены ключевые потоки данных и управления.

Импорт посылок. Преподаватель настраивает интеграцию для курса или задания и запускает импорт вручную либо по расписанию. Интеграционный адаптер обращается к внешнему API, получает список попыток, нормализует данные и сохраняет метаданные в PostgreSQL. Исходные

файлы могут храниться в базе данных или в файловом хранилище (выбор зависит от объёма данных и требований к резервному копированию). Для предотвращения дубликатов используется внешний идентификатор посылки и/или хеш содержимого.

Проверка на заимствования. После импорта система формирует набор посылок для задания и инициирует проверку во внешнем сервисе сравнения программ через API. Компонент **Plagiarism Service** подготавливает пакет входных данных (набор файлов, язык и метаданные, идентификаторы участников), отправляет запрос на запуск и получает идентификатор задачи. Результат может поступать в виде периодического опроса статуса (polling) или через обратный вызов (webhook) после завершения расчёта. По завершении внешняя система возвращает агрегированные оценки сходства и артефакты отчёта (например, структурированный JSON и/или HTML/PDF с подсветкой совпадающих фрагментов). Эти данные сохраняются в PostgreSQL и становятся доступными для просмотра и группировки в интерфейсе преподавателя.

LLM-анализ. Для каждой посылки (или выбранных подозрительных решений) запускается фоновая задача, которая обращается к внешнему LLM API, получает структурированный ответ и сохраняет его в базе данных. Полученный результат используется для формирования обратной связи и генерации вопросов, уточняющих понимание решения.

Уведомления и отчётность. После завершения импорта и проверок соответствующие события передаются в сервис уведомлений и сервис экспорта. Экспорт может формировать таблицы (например, в Google Sheets), а уведомления — доставлять сообщения в Telegram.

5. Реализация ключевых модулей и интеграций

В данном разделе описаны реализация и проектные спецификации системы в терминах модулей и интерфейсов. В случаях, когда конкретный контракт не подтверждён исходными материалами, он приводится как **возможный вариант реализации**.

5.1. Технологический стек и организация кода

В качестве основы серверной части предполагается использование Python 3.x и FastAPI [4]. Для работы с базой данных применяется PostgreSQL [7] и ORM-подход (SQLAlchemy/SQLModel) с поддержкой миграций (например, Alembic). Выполнение фоновых задач реализуется с помощью Celery [3] и брокера Redis [5]. Для интеграции с LLM используется внешний API сменяемого провайдера. Фронтенд планируется реализовать в виде SPA на React в соответствии с роадмапом разработки.

5.2. Модель данных (проектное предложение)

В таблице 5.1 приведена примерная декомпозиция сущностей доменной модели.

Таблица 5.1 – Ключевые сущности доменной модели (вариант проектирования).

Сущность	Назначение
User	Учётная запись: роль, контактные данные, внешние идентификаторы.
Course	Курс: метаданные, владелец(ы), связь с интеграциями.
Assignment	Задание: сроки, настройки проверки, привязка к внешним задачам.
Submission	Посылка (решение): автор, язык, исходник/ссылка на файл, статус.
IntegrationConfig	Параметры интеграции (тип, токены/ключи, идентификаторы контекста).
PlagiarismReport	Результаты сравнения: пары/кластеры, процент сходства, сегменты.
AIAnalysisReport	Структурированный ответ LLM: резюме, риски, вопросы, рекомендации.
ExportJob	История экспортов и параметры выгрузки (например, таблица, диапазон).
AuditLog	Запись аудита: действие, субъект, объект, время, метаданные.

При реализации важно предусмотреть хранение связей «курс—задание—посылка» и статусов обработки, обеспечить идемпотентность импорта за счёт уникальности внешних идентификаторов, поддерживать версионирование отчётов при повторных запусках проверок, а также вести аудит действий преподавателя и системных событий.

5.3. Примерная спецификация API (проектное предложение)

Ниже приведён возможный набор REST-эндпоинтов, ориентированный на сценарии преподавателя и студента. Форматы указаны как рекомендация и могут быть уточнены в ходе реализации.

Аутентификация.

```
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
GET /api/v1/auth/me
```

Листинг 1 – Примерные эндпоинты аутентификации (вариант)

Курсы и задания.

```
GET /api/v1/courses
POST /api/v1/courses
GET /api/v1/courses/{course_id}
PATCH /api/v1/courses/{course_id}
GET /api/v1/courses/{course_id}/assignments
POST /api/v1/courses/{course_id}/assignments
GET /api/v1/assignments/{assignment_id}
PATCH /api/v1/assignments/{assignment_id}
```

Листинг 2 – Примерные эндпоинты управления курсами и заданиями (вариант)

Импорт и посылки.

```
POST /api/v1/assignments/{assignment_id}/import
GET /api/v1/assignments/{assignment_id}/submissions
GET /api/v1/submissions/{submission_id}
```

Листинг 3 – Примерные эндпоинты импорта и доступа к посылкам (вариант)

Запуск проверок и отчёты.

```
POST /api/v1/assignments/{assignment_id}/checks/plagiarism
POST /api/v1/submissions/{submission_id}/checks/ai
GET /api/v1/assignments/{assignment_id}/reports/plagiarism
GET /api/v1/submissions/{submission_id}/reports/ai
```

Листинг 4 – Примерные эндпоинты проверок и отчётов (вариант)

Экспорт и уведомления.

```
POST /api/v1/assignments/{assignment_id}/export
GET /api/v1/exports/{export_id}
GET /api/v1/notifications/stream (SSE/WebSocket)
```

Листинг 5 – Примерные эндпоинты экспорта и уведомлений (вариант)

Для вебхуков внешних систем (при их использовании) рекомендуется проверять подпись или секретный токен в заголовках, обеспечивать строгую идемпотентность по ключу события (`event_id`), а также как можно быстрее возвращать успешный ответ (например, 200 OK), вынося основную обработку в асинхронные фоновые задачи.

5.4. Модуль проверки на заимствования

Проверка решений на заимствования в рамках PlagLens реализуется как интеграция с **внешним сервисом сравнения программ**, доступным по API. Такой подход позволяет использовать существующие методы сопоставления кода и готовые механизмы построения отчётов, не дублируя функциональность специализированных решений. При этом доменная логика платформы не привязывается к конкретному поставщику: взаимодействие с сервисом инкапсулируется за интерфейсом провайдера и при необходимости может быть заменено без изменения пользовательских сценариев и модели данных.

В логической архитектуре выделяется компонент **Plagiarism Service**, который выступает в роли адаптера и оркестратора проверки. Он подготавливает входные данные (набор посылок в рамках задания, метаданные языка и идентификаторы участников), инициирует запуск проверки во внешнем сервисе и сохраняет результат в виде структурированного отчёта. В зависимости от возможностей внешнего API возможны два режима получения результата: (I) периодический опрос статуса по идентификатору запуска (*polling*) и (II) приём обратного вызова (*webhook*) после завершения расчёта.

Для обеспечения надёжности и воспроизводимости рекомендуется фиксировать версию запуска проверки (например, с помощью `run_id` внешнего сервиса и метки времени), поддерживать идемпотентные повторные запросы и ограничивать объём передаваемых данных. Результаты

проверки предлагается хранить в базе данных в виде агрегированных метрик (например, максимальная и средняя степень сходства, число подозрительных пар) и ссылок на детальные артефакты (HTML/PDF-отчёт, JSON со списком совпадений). Конкретный формат отчёта зависит от возможностей выбранного сервиса; пример возможного контракта приведён в приложении Б.

Важно отметить, что внутренние алгоритмы сопоставления кода (fingerprinting, AST-методы и другие) в данном варианте находятся на стороне внешнего сервиса. Их описание используется в обзоре литературы как обоснование применимости подходов, однако не рассматривается как часть вычислительного ядра, реализуемого в PlagLens.

5.5. Модуль LLM-анализа кода

LLM-анализ рассматривается как вспомогательный контур системы. Предлагается получать структурированный результат в формате JSON, содержащий краткое резюме решения, перечень потенциально подозрительных признаков (при их наличии), набор вопросов для устной проверки понимания, а также рекомендации по улучшению читаемости и безопасности кода.

Проектное допущение (пример контракта).

```
{
  "summary": "...",
  "risk_signals": [
    {"type": "style_jump", "severity": "medium", "details": "..."},
    {"type": "generic_solution", "severity": "low", "details": "..."}
  ],
  "questions": [Почему
    "выбран именно такой алгоритм?", Что
    "будет при граничных входных данных?"
  ],
  "recommendations": [Добавить
    "проверку входных параметров...", Упростить
    "ветвление в функции..."
  ]
}
```

Листинг 6 – Пример формата AI-отчёта (вариант JSON)

Указанный результат может быть получен через вызовы внешнего API LLM-провайдера. В целях идемпотентности и контроля затрат рекомендуется кэшировать результаты и ограничивать повторные вызовы.

5.6. Интеграции со сторонними сервисами

Stepik. Интеграция может выполняться с использованием OAuth-токена и периодического импорта посылок через Stepik API [11]. Для снижения нагрузки целесообразно использовать инкрементальные выборки по времени и хранить курсор (метку времени последней импортированной посылки).

Яндекс.Контест. Сценарий интеграции аналогичен и предполагает хранение идентификатора контеста и параметров доступа. В случае отсутствия прямого API возможен импорт через выгрузку результатов в формате CSV или JSON как альтернативный канал (проектное допущение).

Telegram-уведомления. Сервис уведомлений может отправлять сообщения преподавателю о завершении проверок и обнаруженных подозрительных случаях через Telegram Bot API [12].

Экспорт в Google Sheets. Экспорт результатов курса или задания может быть реализован через Google Sheets API [6] с использованием сервисного аккаунта.

6. Безопасность и управление секретами

Безопасность PlagLens рассматривается на нескольких уровнях и включает аутентификацию и авторизацию пользователей, защиту секретов интеграций, конфиденциальность данных и устойчивость инфраструктуры к типовым атакам.

6.1. Аутентификация и авторизация

Базовый вариант предполагает использование JWT-аутентификации с разделением ролей (администратор, преподаватель, студент) и проверкой прав доступа на уровне gateway и доменных сервисов. Дополнительно возможно подключение внешней авторизации (например, через OAuth2), однако для минимальной версии системы это не является обязательным.

6.2. Хранение секретов

Секреты внешних интеграций (API-ключи, OAuth-токены, ключи сервисных аккаунтов) не должны храниться в открытом виде. Предпочтительным является использование защищённого хранилища секретов (Vault/Secrets) и загрузка секретов в окружение сервисов на этапе деплоя.

6.3. Конфиденциальность и защита данных

При развёртывании системы рекомендуется обязательное использование HTTPS (например, на уровне Nginx или Traefik) и исключение утечек чувствительных данных через логи и ответы API. При обработке исходного кода как недоверенных данных важно избегать его исполнения внутри сервисов (при отсутствии семантического анализа). Для защиты от злоупотреблений целесообразно применять лимиты запросов и контроль частоты запуска проверок.

7. Наблюдаемость, мониторинг и уведомления

Наблюдаемость необходима для стабильной эксплуатации платформы и включает сбор метрик, логов и настройку алертов.

7.1. Метрики и мониторинг

В PlagLens целесообразно реализовать endpoint метрик (например, `/metrics`) в формате Prometheus [2]. К ключевым метрикам относятся время ответа API (например, `p50/p95`) и распределение ошибок по кодам, длина очередей фоновых задач и время выполнения импортов и проверок, объём импортированных посылок и доля дедупликации, а также число выявленных подозрительных пар или кластеров. При наличии учёта затрат на LLM дополнительно целесообразно фиксировать число запросов и объём потребляемых токенов.

На основе собранных метрик строятся дашборды в Grafana [8] и настраиваются алерты, например на рост числа ошибок интеграций или увеличение времени обработки задач.

7.2. Логи и трассировка

Для ключевых потоков (импорт, проверка, экспорт) рекомендуется использовать структурированное логирование с корреляционными идентификаторами (`request_id`, `job_id`). При необходимости трассировка может быть расширена до распределённой с использованием OpenTelemetry (проектное предложение).

8. Тестирование

В отсутствие детальных численных метрик в исходных материалах тестирование рассматривается как методика, применимая к PlagLens.

8.1. Уровни тестирования

Тестирование целесообразно организовать на нескольких уровнях. **Модульные тесты** покрывают подготовку входных данных для проверок, правила RBAC и адаптеры внешних сервисов, а также преобразование и сохранение отчётов. **Интеграционные тесты** проверяют API (FastAPI), взаимодействие с PostgreSQL и очередями фоновых задач. **Контрактные тесты** фиксируют ожидаемые форматы ответов внешних API и обработку ошибок и таймаутов. Дополнительно проводится **сценарное ручное тестирование** критических пользовательских сценариев: настройка курса, импорт посылок, запуск проверок, просмотр отчётов и экспорт результатов.

8.2. Критические сценарии

К критическим сценариям относятся корректное разграничение доступа между студентом и преподавателем, идемпотентный импорт посылок без появления дубликатов, воспроизводимость результатов проверки на заимствования при повторных запусках, устойчивость к отказам внешних API (повторы запросов и деградация функциональности), а также корректность экспорта и отсутствие утечек приватных данных.

8.3. Ограничения результатов

При отсутствии возможности собрать численные метрики в ходе разработки (например, precision/recall для детекции плагиата или время обработки на реальных курсах) это следует фиксировать как ограничение и описывать план последующих измерений: выбор эталонной разметки, протокол эксперимента и подходы к валидации.

9. План выполнения работ и связь с роадмапом

Проект реализуется в виде двухнедельных спринтов с резервным завершающим этапом. Использование внешних API для проверки заимствований и LLM-анализа обеспечивает сменяемость провайдеров без изменения доменной логики.

В таблице 9.1 приведена сводка этапов работ и показано, как элементы плана соотносятся с разделами отчёта.

Таблица 9.1 – Сводка плана работ PlagLens по периодам (кратко).

№	Период	Ключевые результаты и задачи
1	Нед. 1 — 2	Подготовка проекта: репозиторий, базовая архитектура, PostgreSQL и миграции, каркас FastAPI, контур фоновых задач (очереди и воркеры).
2	Нед. 3 — 4	Пользователи и роли: аутентификация и выдача JWT, базовые профили и права, проверки доступа на уровне API; CRUD курсов и заданий, хранение исходников и метаданных.
3	Нед. 5 — 6	Интеграции импорта: подключение источников по API, OAuth (при необходимости), импорт посылок, дедупликация и идемпотентность, статусы обработки.
4	Нед. 7 — 8	Проверка заимствований: интеграция с внешним сервисом по API, запуск проверок, получение результатов, хранение и отображение отчётов.
5	Нед. 9 — 10	LLM-анализ: интеграция с внешним провайдером по API, проработка промптов и формата результата, сохранение замечаний и отображение слоя рекомендаций.
6	Нед. 11 — 12	Интерфейс преподавателя: обзор посылок, запуск проверок, просмотр отчётов, комментарии и публикация итоговой обратной связи.
7	Нед. 13 — 14	Кабинет студента: статусы проверок и итоговая обратная связь, обеспечение приватности результатов, администрирование и аудит.
8	Нед. 15 — 20	Завершение проекта: экспорт и дашборды, наблюдаемость, стабилизация и регрессионное тестирование, развёртывание и эксплуатационная документация, финальная вычитка и подготовка к сдаче.

10. Заключение

В данной работе рассмотрен проект PlagLens — платформа для автоматизации проверки программных заданий. Показана актуальность задачи для образовательных курсов, в которых объём проверок постоянно растёт, а использование генеративных моделей усложняет ручную экспертизу решений. В работе сформулированы требования и пользовательские сценарии, предложена модульная архитектура с выделением доменных сервисов, интеграционных адаптеров и сервисов анализа. В теоретической части проанализированы подходы к выявлению заимствований (в частности, fingerprinting и winnowing) и практика их применения в специализированных системах.

В проектируемой платформе проверка на заимствования реализуется как интеграция с внешним сервисом по API, а интеграция с LLM рассматривается как вспомогательный контур, предоставляющий структурированную обратную связь и сигналы риска при сохранении сменяемости провайдера. Также предложены модель данных и примерная спецификация API, описаны механизмы безопасности и управления секретами, контур наблюдаемости на базе Prometheus/Grafana, а также методика тестирования.

Ограничения. Для полноценной верификации качества детектирования заимствований и LLM-сигналов необходимы реальный датасет и эталонная разметка, а также сбор метрик производительности на эксплуатационной нагрузке. Дополнительно требуется уточнение возможностей API внешних платформ и юридических аспектов хранения решений.

Направления развития. В качестве перспективных направлений развития можно выделить расширение языко-зависимой нормализации на основе AST, кластеризацию заимствований на уровне групп, обогащение отчётов сравнением с эталонными решениями, более строгие политики конфиденциальности и расширение набора интеграций.

Список литературы

- [1] Alex Aiken. *MOSS (Measure Of Software Similarity)*. URL: <https://theory.stanford.edu/~aiken/moss/> (дата обр. 02.02.2026).
- [2] Prometheus Authors. *Prometheus Documentation*. URL: <https://prometheus.io/docs/> (дата обр. 02.02.2026).
- [3] Celery contributors. *Celery Documentation*. URL: <https://docs.celeryq.dev/> (дата обр. 02.02.2026).
- [4] FastAPI contributors. *FastAPI Documentation*. URL: <https://fastapi.tiangolo.com/> (дата обр. 02.02.2026).
- [5] Redis contributors. *Redis Documentation*. URL: <https://redis.io/docs/> (дата обр. 02.02.2026).
- [6] Google. *Google Sheets API Documentation*. URL: <https://developers.google.com/sheets/api> (дата обр. 02.02.2026).
- [7] PostgreSQL Global Development Group. *PostgreSQL Documentation*. URL: <https://www.postgresql.org/docs/> (дата обр. 02.02.2026).
- [8] Grafana Labs. *Grafana Documentation*. URL: <https://grafana.com/docs/> (дата обр. 02.02.2026).
- [9] Lutz Prechelt, Guido Malpohl и Michael Philippsen. «Finding Plagiarisms among a Set of Programs with JPlag». В: *Journal of Universal Computer Science*. Т. 8. 11. 2002, с. 1016—1038.
- [10] Saul Schleimer, Daniel S. Wilkerson и Alex Aiken. «Winnowing: Local Algorithms for Document Fingerprinting». В: *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*. ACM, 2003, с. 76—85.
- [11] Stepik. *Stepik API Documentation*. URL: <https://stepik.org/api/docs/> (дата обр. 02.02.2026).
- [12] Telegram. *Telegram Bot API*. URL: <https://core.telegram.org/bots/api> (дата обр. 02.02.2026).
- [13] Yandex. *Yandex Contest: API/Integration Documentation*. URL: <https://contest.yandex.ru/> (дата обр. 02.02.2026).

А. Список сокращений

API Application Programming Interface.

AST Abstract Syntax Tree.

BFF Backend for Frontend.

JWT JSON Web Token.

LLM Large Language Model.

OAuth Open Authorization.

RBAC Role-Based Access Control.

SSE Server-Sent Events.

WebSocket WebSocket protocol.

Б. Пример формата отчёта по заимствованиям (проектное допущение)

Ниже приведён возможный состав полей для отчёта антиплагиата на уровне задания. На практике внешний сервис может возвращать как агрегированные метрики (например, максимальное сходство и число подозрительных пар), так и подробные данные по каждой паре. Чтобы снизить зависимость от поставщика, имеет смысл хранить нормализованную структуру (идентификаторы, метрики, диапазоны строк совпадений) и ссылки на «сырые» артефакты, которые отдаёт сервис (HTML/PDF, исходный JSON). Пример ниже показывает один из возможных вариантов результата в формате JSON.

```
{
  "assignment_id": 123,
  "run_id": "plg_8b7c1f2d",
  "status": "completed",
  "summary": { "submissions_total": 42, "pairs_suspected": 1, "max_similarity":
    0.82 },
  "pairs": [
    { "a": 1001, "b": 1008, "sim": 0.82, "fragments": [{ "a": [10, 35], "b": [12,
      37] }], "evidence": "rep:plg_8b7c1f2d" }
  ]
}
```

Листинг 7 – Пример структуры отчёта по заимствованиям (вариант JSON)